

FLIPPER BLACK

Reading Datasheets + KiCad 10 Schematic Guide

From zero to complete schematic — step by step

This guide teaches you two things: how to extract everything you need from a chip datasheet, and how to turn that into a correct schematic in KiCad 10. It is written specifically for Flipper Black. Every concept is explained before you are asked to do it.

Contents

- Part 1** How to Read a Datasheet for Schematic Design
- Part 2** KiCad 10 — Project Setup and Interface
- Part 3** Adding Symbols and Creating Custom Ones
- Part 4** Wiring the Schematic Correctly
- Part 5** Power Symbols, Decoupling Caps, and Net Labels
- Part 6** Hierarchical Sheets — Splitting a Big Design
- Part 7** Running ERC and Fixing Errors
- Part 8** Flipper Black — Schematic Order of Operations

Part 1 — How to Read a Datasheet for Schematic Design

A datasheet is a chip manufacturer's official document. It tells you everything you need to wire the chip correctly. Most beginners skip it. That is why their boards don't work. This section teaches you what to extract from any datasheet before opening KiCad.

The 7-step reading order

Every datasheet is different but every useful datasheet has these seven sections. Read them in this order every time.

1	Step 1 — Typical Application Circuit	Go here first. It is a complete reference schematic showing the chip connected with all its required external components. Copy it exactly to start. This is your baseline. Only deviate when you have a specific reason.
2	Step 2 — Features list (page 1)	Bullet list of what the chip can do. Confirm: frequency range, interface (SPI/I2C/UART), voltage range, and any specific feature you are counting on. If it is not in this list, the chip does not have it.
3	Step 3 — Pin description table	Lists every pin by name, number, direction (input/output/power), and function. This is the most-used page in the whole document. You will return to it constantly while drawing the schematic.
4	Step 4 — Absolute maximum ratings	The limits you must NEVER exceed — max voltage, max current, max temperature. Exceeding any of these kills the chip instantly or slowly. Design to the Recommended Operating Conditions, not the maximum ratings.
5	Step 5 — Electrical characteristics	The actual operating specs — supply current, logic thresholds, timing. Confirms your LDO output voltage is compatible and your GPIO logic levels match.
6	Step 6 — Register map	For configurable chips like CC1101 and SX1276. A table of every internal register, its address, bits, and default value. You will use this when writing firmware, not schematic design.
7	Step 7 — Errata document	A SEPARATE document from the manufacturer. Official list of known silicon bugs. Always download and read errata before finalizing your design. Espressif, TI, and NXP all publish errata.

What to extract and write down before KiCad

Before drawing a single wire in KiCad, extract these four things from each chip's datasheet and write them in a notebook or text file:

What to extract	Where to find it	Why you need it
Supply voltage (VCC range)	Electrical characteristics	Confirm your 3.3V rail is in range

What to extract	Where to find it	Why you need it
All VCC and GND pin numbers	Pin description table	Every VCC pin needs a decoupling cap in schematic
Interface pins (MOSI/MISO/SCK/CS or SDA/SCL or TX/RX)	Pin description table	Map to your GPIO pin assignment document
Required external components	Typical application circuit	Crystal, decoupling caps, pull-ups — all listed here
Logic voltage level	Electrical characteristics / Absolute max	Confirms if level shifting is needed (e.g. GPS at 1.8V)
Active HIGH or active LOW pins	Pin description table	CS, RESET, IRQ — direction matters for pull-up/pull-down

Decoupling capacitors — what they are and why they are mandatory

Every chip draws sudden bursts of current when it switches states. Without decoupling caps, these bursts cause voltage dips on the power rail that corrupt data and crash chips. Decoupling caps are local energy reservoirs — tiny batteries sitting right next to each chip.

Cap value	Purpose	Placement
100 nF (0.1 uF) ceramic X5R/X7R	High-frequency decoupling — absorbs fast current spikes	One per every VCC pin, as close as physically possible
1-10 uF ceramic X5R	Mid-frequency bulk storage	One per chip, slightly further away from VCC pin
100 uF electrolytic	Low-frequency bulk on main power rail	Near LDO output, one per board

CRITICAL: NEVER use Y5V dielectric capacitors for decoupling. At 3.3V operating voltage, a Y5V '100nF' cap becomes approximately 18nF — it is basically useless. Always use X5R or X7R dielectric. The dielectric code is in the part description on LCSC.

How to read the CC1101 datasheet — worked example

Open the CC1101 datasheet from TI (ti.com/lit/ds/symlink/cc1101.pdf). Here is exactly what to extract:

What you find	Value	What it means for your schematic
Typical application circuit	Page 1 of datasheet	Copy the crystal + decoupling cap arrangement exactly
VCC range	1.8V to 3.6V	3.3V rail is fine
XTAL frequency	26 MHz required	Buy a 26MHz crystal — not 27MHz, not 25MHz
Interface	SPI active LOW CS	CS pin needs 10k pull-up to 3.3V
GDO0 pin	Configurable output	Connect to ESP32 GPIO — used for TX/RX state signal to RF switch
GDO2 pin	Configurable output	Connect to ESP32 GPIO — used for packet received interrupt
Package	QFN-20, 4x4mm	Select QFN-20 footprint in KiCad

TIP: For CC1101: use TI SmartRF Studio 7 (free download from ti.com) to generate the register values for your target frequency. Never calculate CC1101 register values manually — the formula involves fractional PLLs and is error-prone.

Part 2 — KiCad 10 Setup and Interface

KiCad 10 was released April 2025. Download it from kicad.org. The official documentation for KiCad 10 is at docs.kicad.org/10.0/en/. Always use the official docs — they are accurate and complete.

IMPORTANT: One experienced designer noted KiCad 10 has some bugs in early releases. If you encounter instability, KiCad 9 is fully stable and the workflow is identical. Everything in this guide applies to both versions.

First time setup

1	Install KiCad 10	Download from kicad.org . On Windows, run the installer. On first launch, select 'Install default libraries' — this gives you thousands of ready-made symbols and footprints.
2	Create a new project	File > New Project. Give it a name: 'Flipper_Black'. KiCad creates a folder with a .kicad_pro file, a .kicad_sch schematic file, and a .kicad_pcb board file. Keep all files in this folder.
3	Open Schematic Editor	Double-click the .kicad_sch file in the project window, or click the Schematic Editor button. This is where you will spend most of your time.
4	Set page size	File > Page Settings. Set paper size to A3 or A2 — Flipper Black has many components and needs space. Add your name and project name in the title block.

The KiCad 10 schematic editor — what each part does

Area	What it is	How to use it
Canvas	The white drawing area where your schematic lives	Scroll to zoom. Middle-click drag to pan. G key to grab and move.
Right toolbar	Place tools — symbols, wires, labels, power ports	Click or use keyboard shortcuts (A = add symbol, W = wire, P = power port)
Left toolbar	View tools — zoom, grid, highlight	Mostly use keyboard shortcuts instead
Properties panel	Shows properties of selected item	Click any component to see and edit its reference, value, footprint
Net Inspector (Inspect menu)	Lists all nets and connections	Use to verify every pin is connected before ERC

Essential keyboard shortcuts — memorize these

Key	Action
A	Add symbol (place a component)
W	Draw wire
P	Add power port (VCC, GND, +3V3)

Key	Action
L	Add net label
R	Rotate selected item
G	Grab and move (keeps connections)
E	Edit properties of selected item
Delete	Delete selected item
Ctrl+Z	Undo
Ctrl+S	Save
Ctrl+D	Duplicate
F	Flip (mirror) selected item
Scroll wheel	Zoom in/out
Middle click drag	Pan the canvas

Part 3 — Adding Symbols and Creating Custom Ones

A symbol in KiCad is the schematic representation of a component — a box with pins. It is NOT the physical footprint. The symbol shows logical connections. The footprint shows physical pads on the PCB. You assign a footprint to each symbol later.

Adding a symbol from the built-in library

1	Press A	Opens the 'Choose Symbol' dialog. A search box appears.
2	Search by name or part number	Type 'ESP32' or 'CC1101' or 'resistor'. KiCad searches all installed libraries. For generic passives, search 'R' for resistor, 'C' for capacitor.
3	Select the correct symbol	Click it once to preview. Verify the pin names match your datasheet. Click OK.
4	Place it on the canvas	Move mouse to position, click to place. Press Escape when done placing.
5	Assign footprint	Double-click the symbol > Properties > Footprint field. Click the browse button. Search for the correct package (e.g. QFN-20).

TIP: For Flipper Black chips, most symbols exist in the default KiCad library. Search 'ESP32-S3', 'CC1101', 'SX1276', 'PN532'. If not found, use SnapEDA (snapeda.com) to download a KiCad symbol + footprint pair for free.

Creating a custom symbol (when the chip is not in the library)

If a chip is not in KiCad's library or SnapEDA, you create it manually in the Symbol Editor. This takes 15-30 minutes per chip. Here is the process:

1	Open Symbol Editor	In the KiCad project window: Tools > Symbol Editor. Or from the schematic editor: Place > Add Symbol > Symbol Editor button.
2	Create new library	File > New Library. Name it 'Flipper_Black'. Save it in your project folder. This keeps your custom symbols with your project.
3	Create new symbol	File > New Symbol. Enter the chip name (e.g. 'MAX-M8Q'). Set reference prefix to 'U'.
4	Add pins from datasheet	Place > Add Pin. For every pin in the datasheet pin table: set pin name, pin number, and direction (Input/Output/Bidirectional/Power). Pin numbers must match the datasheet exactly.
5	Draw the body	Place > Add Rectangle. Draw a box around the pins. Pins on left = inputs. Pins on right = outputs. Power pins on top/bottom.
6	Save and use	File > Save. Return to schematic editor. Press A, search your chip name. It appears in your custom library.

CRITICAL: Pin numbers in KiCad must match the physical pin numbers in the datasheet exactly. If pin 1 in KiCad is connected to a different net than pin 1 on the physical chip, your board will not work. Always cross-check.

Part 4 — Wiring the Schematic Correctly

Wiring in KiCad connects pins together to form nets. A net is a named group of connections — like 'SPI2_MOSI' or 'GND'. Every wire you draw is part of a net. There are three ways to connect pins in KiCad.

Three ways to connect pins

Method	When to use it	How
Direct wire (green line)	Pins that are physically close to each other on the canvas	Press W, click pin, route to destination, click to end
Net label	Pins far apart, or same signal on different parts of schematic — like SPI2_MOSI going to 3 chips	Press L, type label name (e.g. SPI2_MOSI), place on wire end. Any two labels with the same name are connected.
Power port	VCC, GND, +3V3 — global power nets that every chip shares	Press P, search 'GND' or '+3V3', place on power pin

IMPORTANT: For Flipper Black, use net labels for ALL SPI signals, UART signals, and inter-chip connections. Do NOT run long wires across the schematic — the canvas becomes unreadable. Label everything.

Naming net labels — follow this convention for Flipper Black

Net	Label name
SPI2 MOSI (CC1101 + PN532 + Display)	SPI2_MOSI
SPI2 MISO	SPI2_MISO
SPI2 SCK	SPI2_SCK
SPI3 MOSI (SX1276 + Flash)	SPI3_MOSI
SPI3 MISO	SPI3_MISO
SPI3 SCK	SPI3_SCK
CC1101 chip select	CC1101_CS
SX1276 chip select	SX1276_CS
PN532 chip select	PN532_CS
Display chip select	DISP_CS
GPS UART to ESP32	GPS_TX
GPS UART from ESP32	GPS_RX
LoRa interrupt	LORA_DIO0
RF switch control	RF_SW_CTRL

TIP: Consistent naming prevents ERC errors and makes your schematic readable. Every label name must match exactly — 'SPI2_MOSI' and 'spi2_mosi' are different nets in KiCad. Use all caps with underscores.

Part 5 — Power Symbols, Decoupling Caps, and Net Labels

Power symbols are special symbols in KiCad that represent global power nets. Every component that connects to GND uses the same GND power symbol. Every component that connects to 3.3V uses the same +3V3 symbol. They are automatically connected to each other without drawing wires.

Power symbols used in Flipper Black

Symbol name	Voltage	Connected to
+3V3	3.3V	ESP32-S3 VCC, CC1101 VCC, SX1276 VCC, PN532 VCC, display, IR, most chips
+1V8	1.8V	MAX-M8Q VCC and VCC_IO only — comes from TLV70018 LDO
GND	0V / Ground	Every chip GND pin, decoupling cap bottom rail, LDO GND
VBAT	3.7V nominal (battery)	TP4056 input, DW01A, battery positive terminal
+VBUS	5V USB input	TP4056 VIN from USB-C connector

How to place decoupling caps in KiCad correctly

1	Identify all VCC pins	Look at the chip symbol. Every pin labeled VCC, VDD, AVCC, DVCC needs its own 100nF decoupling cap.
2	Place capacitor symbol next to chip	Press A, search 'C', place a capacitor symbol directly beside the chip's VCC pin on the canvas. Keep it visually close — this reminds you to place it physically close on the PCB later.
3	Connect top of cap to same VCC net	Draw a short wire from capacitor top pin to the chip VCC pin. Then add a +3V3 power symbol on that wire.
4	Connect bottom of cap to GND	Add a GND power symbol to the capacitor bottom pin.
5	Set capacitor value	Double-click cap > Properties > Value field. Type '100nF'. For bulk caps type '10uF'.
6	Set capacitor footprint	Footprint field > Browse > search '0402'. For 10uF use '0603'.

Flipper Black decoupling cap count: ESP32-S3 has 9 VCC pins = 9x 100nF caps. CC1101 has 2 VCC pins = 2x 100nF. SX1276 = 1x 100nF + 1x 10uF. PN532 = 1x 100nF. MAX-M8Q = 2x 100nF. Total: approximately 20-25 decoupling caps on the schematic. This is normal.

Part 6 — Hierarchical Sheets — Splitting a Big Design

Flipper Black has 8+ chips. Putting everything on one schematic sheet makes it unreadable. KiCad supports hierarchical sheets — a parent sheet with sub-sheets, each containing one section of the design. This is exactly how the Flipper Zero hardware schematics are organized.

Recommended sheet structure for Flipper Black

Sheet name	Contents
Main (top level)	Sheet hierarchy overview, power rails, connectors only
MCU	ESP32-S3, W25Q128 flash, crystal, all decoupling, strapping pins, BOOT/EN buttons
Radio_SubGHz	CC1101, 26MHz crystal, RF switch, PA amplifier, antenna connector
Radio_LoRa	SX1276, antenna connector, decoupling
NFC	PN532, NFC antenna coil, decoupling, HSU strapping
GPS	MAX-M8Q, TLV70018 1.8V LDO, TXB0102 level shifter, TCXO, antenna connector, backup supercap
Display	3.5in IPS display connector, touch controller, backlight driver
Power	TP4056 charger, DW01A protection, AP2112K LDO, battery connector, USB-C connector

How to create hierarchical sheets in KiCad 10

1	Place a hierarchical sheet block	In the main schematic: Place > Add Hierarchical Sheet. Draw a rectangle on the canvas. Give it a name (e.g. 'MCU') and a filename (e.g. 'mcu.kicad_sch').
2	Add hierarchical pins	These are the connection points between sheets. Right-click the sheet block > Import Sheet Pin. Or add pins manually via Place > Add Hierarchical Pin.
3	Open the sub-sheet	Double-click the sheet block. A new schematic editor opens for that sheet. Draw the circuit here.
4	Connect sheets with global labels	Use Place > Add Global Label for signals that cross multiple sheets (like GND, +3V3, SPI2_MOSI). Global labels with the same name connect automatically across all sheets.
5	Navigate between sheets	Click the sheet block to enter it. Press Escape or use the breadcrumb at the top to go back to parent.

TIP: Study the Flipper Zero open source schematics at github.com/flipperdevices/flipperzero-firmware — they use hierarchical sheets and are an excellent real-world reference for how to organize a complex design like Flipper Black.

Part 7 — Running ERC and Fixing Errors

The Electrical Rules Check (ERC) is KiCad's automated checker. It scans your schematic for logical errors — unconnected pins, conflicting drivers, missing power flags. Always run ERC and fix all errors before moving to PCB layout. Zero errors and zero warnings is the goal.

How to run ERC

In the schematic editor: Inspect > Electrical Rules Checker > Run ERC. Or click the bug icon in the toolbar. Errors appear as red markers on the schematic. Click any error to jump to it.

Common ERC errors and how to fix them

Error	What it means	How to fix
Pin unconnected	A pin has no wire, label, or power symbol connected	Add a wire to the net, or add a 'No Connect' marker (press Q) if the pin is intentionally unused
Power pin not driven	A power net (VCC, GND) has no source	Add a PWR_FLAG symbol to the net. This tells KiCad the power comes from an external source.
Conflicting net names	Two different labels connected to the same wire	Delete one label or rename to match
Pin connected to pin (no net)	Two component pins connected directly with no net name	Add a net label to the wire between them
Duplicate reference	Two components with the same reference (U1 U1)	Tools > Annotate Schematic > Annotate All. This auto-assigns unique references.
Hierarchical pin not connected	A sheet pin has nothing connected to it inside the sub-sheet	Open the sub-sheet and add a hierarchical label matching the pin name

IMPORTANT: PWR_FLAG is not optional. KiCad requires at least one PWR_FLAG on every power net (+3V3, GND, +1V8, VBAT). Without it, ERC throws 'Power pin not driven' on every chip. Place PWR_FLAG symbols near your LDO outputs and battery connector.

Annotate schematic before ERC

Before running ERC, always annotate: Tools > Annotate Schematic > Annotate All > OK. This assigns sequential reference designators (U1, U2, C1, C2...) to all components. Without annotation, ERC cannot identify components correctly.

Part 8 — Flipper Black Schematic — Order of Operations

This is your exact build sequence. Do not jump ahead. Each step must be complete before the next one starts.

1	Day 1 — Project setup + MCU sheet	Create KiCad project. Set up 8 hierarchical sheets. Draw the ESP32-S3 sheet first — place chip symbol, all decoupling caps (9x 100nF + bulk caps), 40MHz crystal with load caps, CHIP_PU pull-up, GPIO0 pull-up, BOOT button, EN button, W25Q128 flash with full SPI connections. Add all net labels for every GPIO. Run ERC on this sheet alone before continuing.
2	Day 2-3 — Power sheet	Draw TP4056 charger circuit from datasheet typical application. Add DW01A protection. Add AP2112K-3.3 LDO. Add TLV70018 1.8V LDO for GPS. Add battery connector (JST-PH 2-pin). Add USB-C connector. Add all PWR_FLAG symbols. Run ERC.
3	Day 4 — CC1101 + RF switch sheet	Draw CC1101 from TI typical application circuit. Add 26MHz crystal + load caps. Add PA amplifier. Add PE4259 RF switch with GPIO control from CC1101 GDO0. Add SMA antenna connector. Add all SPI net labels matching MCU sheet. Run ERC.
4	Day 5 — SX1276 LoRa sheet	Draw SX1276 from Semtech typical application. Add decoupling. Connect DIO0 to interrupt GPIO. Add SPI net labels for SPI3 bus. Add SMA antenna connector. Run ERC.
5	Day 6 — PN532 NFC sheet	Draw PN532. Hardwire HSU1=GND and HSU0=VCC via 0-ohm resistors (not copper) for SPI mode selection. Add NFC antenna coil circuit from NXP application note AN10609 — this is a matched LC network, copy it exactly. Run ERC.
6	Day 7 — GPS sheet	Draw MAX-M8Q. Add TLV70018 1.8V LDO. Add TXB0102 level shifter (OE tied HIGH to 3.3V, A-side to 1.8V GPS, B-side to 3.3V ESP32 UART). Add 26MHz TCXO from u-blox approved list. Add U.FL antenna connector. Add backup supercap on V_BCKP. Run ERC.
7	Day 8 — Display + IR sheet	Draw 3.5in display connector with SPI + I2C touch pins. Draw IR TX circuit: GPIO > NPN transistor (2N3904) > IR LED array. Draw IR RX: TSOP38238 demodulator with decoupling. Run ERC.
8	Day 9 — Full ERC + review	Run ERC on all sheets. Zero errors, zero warnings. Review every net label — confirm SPI2 labels on MCU sheet match SPI2 labels on CC1101 and PN532 sheets. Confirm SPI3 labels match SX1276 and flash. Review pin assignment document against schematic.
9	Day 10 — Footprint assignment	Double-click every component. Assign correct KiCad footprint: QFN-56 for ESP32-S3, QFN-20 for CC1101, LCC-18 for MAX-M8Q, etc. Verify pad counts match pin counts.

After schematic is complete

Step	Action
Generate netlist	File > Export > Netlist. This file links your schematic to the PCB editor.
Update PCB from schematic	Open PCB editor. Tools > Update PCB from Schematic. All footprints appear as a pile — ready for placement.
Push to GitHub	git add, git commit, git push. Commit after every major section is complete.